

wp-02-heartscale-coherence-rejection-sampling

HeartScale Coherence Rejection Sampling: Inference-Time Alignment via Hidden-State Geometry

AsAManThinks Research — Working Paper WP-02

Weslyn Whitehead Jr. (Docwes / Yare Ace NoK)

ORCID: <https://orcid.org/0009-0005-7707-3210>

May 2026 · MaiaM Alchemist v0.4.x · **Revision 2 (June 2026)**

DOI: [10.5281/zenodo.19600795](https://doi.org/10.5281/zenodo.19600795) (foundations anchor)

Revision 2 changes: (C1) the coherence score is now the **multiplicative, zero-veto** aggregator of Foundations Paper I, presented as the `p=0` member of a single Hölder power-mean family (the prior weighted-harmonic recommendation becomes the `p=-1` member); this matches the public `heartscale-gate` reference implementation. (C1) the capacity budget is stated as `agent_frequency = RI · BC · 64`, tying it to the 64-state frequency scale. (C8) drift uses the product Fisher–Rao distance (L2 in the small-step limit). (C5) drift is decomposed against the TERA conservation surface, adding a conservation-violation channel. No other content changed.

Abstract

We present **HeartScale Coherence Rejection Sampling (HCRS)**, a per-token acceptance filter for autoregressive language models that enforces alignment constraints without a reward model, human preference data, or RLHF. HCRS accepts a candidate token if and only if its **coherence cost** — a multiplicative composite of the token’s TERA-projected hidden-state drift and its entropy excess above a rolling baseline — divided by a session-level capacity budget, does not exceed 1.0. Rejected tokens trigger progressive-temperature resampling from the same distribution; persistent rejection triggers clean EOS truncation. The mechanism is a generalization of the binary `IsEvenlyYoked` predicate from the MaiaM Platform’s C# `HeartScaleValidator`, extended from a one-shot output gate to a per-token generation-time filter, and is consistent with the multiplicative (zero-veto) evaluation of AAMT Foundations Paper I. HCRS is purely inference-time, adds no training cost, and is complementary to RLHF — the rejection trace can be used directly as a reward signal.

1. Motivation

RLHF (Ouyang et al., 2022) and its variants (DPO, PPO, RLAIF) are the dominant paradigm for post-training alignment. They share a common structure: collect human preference data, train a reward model, optimize the

policy against the reward model. This pipeline has well-documented costs: annotation expense, reward model overfit, KL divergence constraints, and training instability.

A complementary question is under-explored: **can we enforce alignment constraints at inference time, using only the model’s own hidden-state geometry, without any learned reward model?**

Contrastive decoding (Li et al., 2023) uses the contrast between a large and small model to suppress undesired tokens. FUDGE (Yang and Klein, 2021) uses a lightweight classifier over the full vocabulary. Speculative decoding (Chen et al., 2023) uses a draft model for efficiency. None of these use the primary model’s own internal geometry as the acceptance criterion.

HCRS uses exactly this. The coherence cost is computed from the shift in the model’s own TERA-projected hidden state induced by a candidate token, combined with that token’s distributional entropy relative to a local baseline. No external model, classifier, or preference data is required.

2. Background: The Ma’at Predicate

In the MaiaM Platform’s Unity C# codebase, agent actions are gated by:

```
bool IsEvenlyYoked(float agentFrequency, float actionWeight) {
    return actionWeight / agentFrequency <= 1.0f;
}
```

This is a **balance constraint**: an action’s weight relative to the agent’s capacity must not exceed 1. The metaphor is the Egyptian Ma’at — the feather of truth on the scale.

The constraint has one binary output per action. HCRS generalizes it along two axes:

1. **Per-token**: applied to every generated token, not just to the final action.
2. **Geometric**: the “actionWeight” is now derived from the model’s own hidden-state dynamics, not a pre-specified float.

3. The Coherence Cost

3.1 Two component signals

Vortex Drift: At each token position, the model maintains a hidden state $h_t \in \mathbb{R}^D$. We define the TERA projection of any hidden state as:

$$\text{TERA}(h) = \text{sigmoid}(W_{\text{proj}} \cdot h + b_{\text{proj}}) \in [0, 1]^4$$

where W_{proj} is the VortexGate's 4D linear layer (shared with routing). The vortex drift for a candidate token p_t is the **product Fisher–Rao distance** between successive TERA coordinates:

$$\begin{aligned} \text{vortex_drift}(p_t, h_t) &= d_{\text{FR}}(\text{TERA}(h_{t+1}(p_t)), \text{TERA}(h_t)) \\ d_{\text{FR}}(u, v) &= \sqrt{(\Sigma_d (2 \cdot \arcsin u_d - 2 \cdot \arcsin v_d)^2)} \end{aligned}$$

where $h_{t+1}(p_t)$ is the next hidden state conditioned on p_t . The Fisher–Rao distance is the natural metric on the product of Bernoulli marginals that define the Meji measure (WP-01 §2.2); it reduces to the Euclidean norm $\|\Delta \text{TERA}\|_2$ in the small-step limit near $v_d = \frac{1}{2}$. Large drift means the candidate token moves the model's archetype substantially; small drift means the token is coherent with the current trajectory.

The drift is decomposed against the TERA conservation surface Σ_κ (WP-01 §2.1a): the component **tangent** to Σ_κ is ordinary archetype drift, while the component **normal** to Σ_κ is a conservation-violation signal. The normal component is folded into the cost with weight ω_n (default absorbed into the geometric weight ω below).

Entropy Excess: The candidate token's distributional uncertainty relative to a rolling baseline:

$$\text{entropy_excess}(p_t) = H(\text{logits}_t) - \text{rolling_mean}(H(\text{logits}_{t-32:t}))$$

where $H(\cdot)$ is the distribution entropy. Positive excess signals an off-manifold generation — the model is less certain than usual, suggesting a potential hallucination or incoherent completion. (Negative excess is clamped to a small positive floor ϵ so the multiplicative score below is well-defined.)

3.2 Composite cost (multiplicative, zero-veto)

The coherence cost combines both signals. We use a single **Hölder power-mean family** parameterized by an exponent p that sets alignment aggressiveness:

$$\text{score}_p(p_t) = (\omega \cdot \text{drift}^p + (1-\omega) \cdot \text{excess}^p)^{1/p} \quad \omega = 0.6 \text{ default}$$

with $p=1$ arithmetic (symmetric sensitivity), $p=-1$ harmonic, and $p \rightarrow 0$ the weighted geometric mean. **We adopt the weighted geometric mean ($p \rightarrow 0$):**

$$\text{score}(p_t) = \text{drift}(p_t)^\omega \cdot \text{excess}(p_t)^{(1-\omega)}$$

This is the **zero-veto evaluator** of AAMT Foundations Paper I (Multiplicative Evaluation): if *either* signal is near zero the score is near zero and the token is accepted, so a token that is geometrically coherent in either dimension passes. This is the AND-biased, safety-conservative behavior we want — it is harder to reject than with the arithmetic mean — and it is now consistent with the multiplicative form used throughout the AAMT framework (the Master Equation $M(t) = \Psi_0 \cdot I \cdot S \cdot C \cdot B \cdot 0 \cdot e^{\{(P-R)t\}}$, the manifestation product $P = C \cdot I \cdot B \cdot A \cdot T$, and the $T \cdot E \cdot R \cdot A$ conservation law). The weighted harmonic mean recommended in Revision 1 is the $p=-1$

member of the same family and exhibits the same AND-bias; the geometric mean is preferred because it is the canonical zero-veto operator and matches the public reference implementation (`heartscale-gate`), where `action_weight = -log p(token) · semantic_load(token)` is likewise a product of a surprisal term and a cost term.

3.3 Note on signal scaling

`drift` and `excess` are normalized to comparable `[ε, ·]` ranges by their rolling baselines before the geometric mean is taken, so neither dimension dominates purely by units. The clamp floor `ε` (default `1e-3`) bounds the score away from a degenerate zero when a signal is exactly zero.

4. The Accept/Reject Loop

```
while t < max_new_tokens:

    # 1. Sample from current breath-phase budget
    p_t = sample(logits_t, T=phase.temperature, top_p=phase.top_p)

    # 2. Score (multiplicative zero-veto cost)
    score = coherence_score(p_t, h_t)          # drift^ω · excess^(1-ω)
    ratio = score / agent_frequency

    # 3. Accept or retry
    if ratio ≤ 1.0:
        emit(p_t)
        h_{t+1} ← forward(p_t)
        t += 1
        continue

    # 4. Progressive-temperature resampling
    T_retry = phase.temperature
    for resample_index in range(MAX_RESAMPLES):    # default MAX_RESAMPLES = 4
        T_retry *= 0.85                          # progressive cooling
        p_t = sample(logits_t, T=T_retry)
        score = coherence_score(p_t, h_t)
        if score / agent_frequency ≤ 1.0:
            emit(p_t)
            h_{t+1} ← forward(p_t)
            t += 1
            break
    else:
        # All resamples exhausted – clean truncation
        emit(EOS)
        break
```

4.1 Resampling dynamics

Progressive temperature reduction during resampling concentrates probability mass on high-probability tokens. High-probability tokens tend to have lower drift (they are in-distribution with the prefix) and lower entropy excess (the model is more certain about them). The cooling therefore increases acceptance probability across resamples, which is the desired behavior — we want to find the highest-probability token that is also coherent, not to perpetually reject and truncate.

4.2 Session capacity: `agent_frequency`

The acceptance budget is the product of the two routing-geometry statistics scaled by the 64-state maximum frequency (WP-01 §5; Foundations 64-State system):

$$\text{agent_frequency} = \text{RI} \cdot \text{BC} \cdot 64$$

where `RI` is Resonant Integrity and `BC` is Breath Coherence. With the single-knob convenience form `RI = BC = \sqrt{\text{capacity}}`, this reduces to `agent_frequency = capacity \cdot 64`, recovering the reference implementation's `from_capacity()` constructor. High budget (large `RI \cdot BC`) permits higher-cost tokens — appropriate for creative/generative tasks; low budget tightens the gate — appropriate for high-stakes factual tasks. In the MaiaM Unity deployment, the player's frequency level maps directly as `level \in \{1..64\}`, so `agent_frequency` is literally a point on the 64-state frequency scale. A token is **Evenly Yoked** (Ma'at) iff $\text{score} / \text{agent_frequency} \leq 1$.

4.3 Fail-open safety

If every candidate at a position is rejected across all resamples, the gate restores the original (ungated) logits for that single step — **fail-open** — so decoding never deadlocks, before the EOS-truncation path of the main loop is reached on persistent failure. This guarantees liveness without sacrificing the per-token bound on the common path.

5. Relationship to the Breath Scheduler

HCRS and the breath scheduler are compositional but independent:

- The breath scheduler (WP-03) controls **how** tokens are sampled: temperature, top-p, attention head count, KV cache fraction.
- HCRS controls **whether** a sampled token is accepted.

They compose in a specific order: the breath scheduler produces a candidate, HCRS evaluates it. If HCRS resamples, it does so within the current breath phase budget (but with progressive temperature cooling overlaid). If HCRS triggers EOS, the breath scheduler transitions to EMPTY phase.

This separation of concerns is deliberate: one mechanism controls compute allocation, the other controls alignment. Neither depends on the other's internal state.

6. Telemetry and Observability

Every rejection emits a structured telemetry event:

```
{
  "kind": "heartscale_reject",
  "token": " hallucinate",
  "ratio": 1.83,
  "score": 1.83,
  "vortex_drift": 1.92,
  "entropy_excess": 1.66,
  "resample_index": 2
}
```

This event stream enables:

1. **Per-turn rejection rate tracking** — a proxy for alignment pressure during a session.
 2. **Adversarial eval coverage** — prompts that produce accepted-but-misaligned tokens can be identified by examining the rejection trace.
 3. **DPO signal extraction** — (prompt, accepted_completion) vs. (prompt, rejected_token_sequence) pairs are implicit in the rejection log, usable for direct preference optimization without additional annotation (the same verifier-gated harvesting used by Compile-Gated DPO, WP-09).
-

7. Why This Is Not Contrastive Decoding

Contrastive decoding (Li et al., 2023) computes the log-ratio of logits from a large “expert” model and a small “amateur” model, amplifying differences to suppress undesired tokens. This requires a second model at inference time.

HCRS uses the primary model's own hidden-state geometry. The “contrast” is between the current hidden state and the candidate-token-conditioned hidden state — a within-model, within-forward-pass operation. The cost is one additional projection (the TERA forward) per candidate token.

The conceptual difference is significant: contrastive decoding suppresses tokens the smaller model would prefer over the larger model; HCRS suppresses tokens that shift the model's own archetype geometry beyond a threshold. These are different criteria — HCRS can accept tokens a contrastive decoder would reject (if the shift is small) and reject tokens contrastive decoding would accept (if the shift is large regardless of model size).

8. Formal Properties

Proposition 1 (Bounded Rejection Rate). For a model producing in-distribution outputs with rolling entropy baseline H_o , and drift baseline D_o , the expected rejection rate at unit budget converges to a constant determined by the tails of the drift and excess distributions.

Proposition 2 (Monotone Capacity Sensitivity). For fixed logits, the acceptance probability at budget c is monotonically non-decreasing in c . As $c \rightarrow \infty$, acceptance probability approaches 1. (Immediate: the zero-veto score is fixed per candidate, and score/c is decreasing in c .)

Proposition 3 (EOS Safety Guarantee). Every generation either terminates normally (EOS from the model) or terminates by HCRS-triggered EOS after at most $\text{max_new_tokens} \times (1 + \text{MAX_RESAMPLES})$ forward passes. No infinite loop is possible. The fail-open path (§4.3) further guarantees no single position can stall.

9. Limitations

- 1. Hidden-state access required.** HCRS requires access to the model’s hidden states at every token, which is not available through standard inference APIs. It is a white-box inference method.
 - 2. Per-token forward-pass overhead.** Computing $h_{\{t+1\}}(p_t)$ for each candidate before acceptance requires an extra partial forward pass (or caching the next-token hidden state efficiently). Implementation optimizations (batching candidates, partial layer evaluation) can reduce this cost significantly.
 - 3. Calibration.** The default $\omega = 0.6$, the rolling window of 32 tokens, and the clamp floor ϵ are empirically derived for the MaiaM domain. Different domains may require recalibration, and the Hölder exponent p may be tuned (toward $p=-1$) for stricter AND-biasing.
 - 4. Adversarial robustness.** An adversarial prompt engineered to produce tokens with low drift and low entropy but misaligned content could evade HCRS. Combining HCRS with a lightweight semantic similarity check — or with the transformation-based (sign-inversion) alignment operator — is a direction for future work.
-

10. Conclusion

HCRS demonstrates that alignment constraints can be enforced at inference time using only the primary model’s hidden-state geometry, without reward models, preference data, or additional training. The Ma’at predicate — a balance constraint from ancient wisdom tradition — provides a mathematically precise acceptance criterion, now expressed in the multiplicative zero-veto form that unifies it with the rest of the AAMT framework. The progressive-temperature resampling strategy, with fail-open safety, ensures that acceptance rate increases across retries while never deadlocking. We believe HCRS is an important complement to RLHF-

based alignment, particularly in deployment environments where reward model inference is cost-prohibitive or where alignment requirements evolve faster than training cycles permit.

References

- Ouyang, L. et al. (2022). Training language models to follow instructions with human feedback.
- Li, X. et al. (2023). Contrastive decoding: Open-ended text generation as optimization.
- Yang, K., Klein, D. (2021). FUDGE: Controlled text generation with future discriminators.
- Chen, C. et al. (2023). Accelerating large language model decoding with speculative sampling.
- Whitehead, W. (2026). MaiaM Wings: Consciousness-Aligned Language Architecture. Zenodo. DOI: 10.5281/zenodo.19600795
- AAMT Foundations Papers I–V: `aamt-foundations/foundations.json` (Paper I: Multiplicative Evaluation)